

CTF (CAPTURE THE FLAG) DENGAN PYTHON

CTF berbasis Python biasanya digunakan dalam tantangan keamanan siber, termasuk *Reverse Engineering*, *Cryptography*, *Web Exploitation*, *Forensics*, dan *Pwn (Binary Exploitation)*. Berikut ini beberapa contoh soal Python untuk CTF serta cara pemecahannya.

A. Reverse Engineering (Code Obfuscation)

1. Pengertian Reverse Engineering

Reverse engineering adalah proses menganalisis suatu produk, perangkat lunak, atau sistem untuk memahami cara kerjanya, struktur, dan komponennya. Tujuan utama dari reverse engineering adalah untuk mendapatkan informasi atau dokumentasi yang tidak tersedia atau tidak lengkap. Proses ini sering dilakukan dengan cara membongkar perangkat keras (hardware) atau menelusuri kode program (software) guna memahami mekanisme di dalamnya.

2. Fungsi Reverse Engineering

a. Analisis Keamanan

Digunakan untuk mengidentifikasi kerentanan dalam suatu sistem, perangkat lunak, atau jaringan guna meningkatkan keamanan.

b. Pemeliharaan dan Perbaikan

Membantu memperbaiki perangkat lunak atau perangkat keras yang sudah tidak memiliki dokumentasi atau dukungan teknis dari pengembang aslinya.

c. Kompatibilitas dan Interoperabilitas

Memungkinkan pengembang menciptakan produk atau perangkat lunak yang kompatibel dengan sistem atau perangkat lain.

d. Pengembangan Produk Baru

Digunakan untuk memahami teknologi pesaing dan mengembangkan inovasi baru berdasarkan teknologi yang sudah ada.

e. Pemulihan Kode Sumber (Source Code Recovery)

Berguna ketika kode sumber dari suatu perangkat lunak hilang atau rusak, sehingga bisa direkonstruksi kembali.

f. Penelitian dan Pembelajaran

Membantu peneliti dan mahasiswa dalam mempelajari bagaimana suatu sistem atau algoritma bekerja.

Contoh Soal :

Soal 1.

Kode Python berikut telah diobfuscate. Temukan nilai flag yang tersembunyi di dalamnya.

```
import base64

def check_flag(user_input):
    enc = "RmxhZ3tQeXRob25fQ1RGX2lzX2Z1bn0="
    dec = base64.b64decode(enc).decode()
    return user_input == dec

user_input = input("Enter the flag: ")
if check_flag(user_input):
    print("Correct!")
else:
    print("Wrong flag!")
```

Pemecahan

- Kode ini menggunakan **Base64 encoding** untuk menyembunyikan flag.
- String terenkripsi adalah **"RmxhZ3tQeXRob25fQ1RGX2lzX2Z1bn0="**.
- Decode menggunakan Python:

```
import base64

print(base64.b64decode("RmxhZ3tQeXRob25fQ1RGX2lzX2Z1bn0=").decode())
```

Output:

Flag{Python_CTF_is_fun}

Jawaban: Flag{Python_CTF_is_fun}



B. Cryptography (Caesar Cipher)

1. Pengertian Cryptography

Cryptography adalah ilmu dan teknik untuk mengamankan informasi dengan cara mengubah data asli (plaintext) menjadi data tersandi (ciphertext) sehingga tidak dapat dibaca oleh pihak yang tidak berwenang.

Kriptografi banyak digunakan dalam keamanan jaringan, aplikasi, sistem login, transaksi online, dan ethical hacking.

2. Fungsi Cryptography

Kriptografi memiliki empat fungsi utama dalam keamanan informasi:

a. Confidentiality (Kerahasiaan)

Menjaga agar informasi hanya dapat dibaca oleh pihak yang berhak.

Contoh:

- Password yang disimpan dalam bentuk hash
- Enkripsi pesan WhatsApp
- HTTPS pada website

b. Integrity (Keutuhan Data)

Menjamin data tidak diubah selama proses pengiriman. Jika data berubah, sistem dapat mendeteksinya.

Contoh:

- Hash (MD5, SHA-256)
- Checksum file

c. Authentication (Autentikasi)

Memastikan identitas pengirim atau pengguna benar dan sah.

Contoh:

1. Login username & password
2. Sertifikat digital
3. Public Key Infrastructure (PKI)

d. Non-Repudiation (Anti Penyangkalan)

Mencegah pihak tertentu menyangkal telah melakukan transaksi atau pengiriman data.

Contoh:

- Tanda tangan digital
- Transaksi perbankan online

Contoh Penggunaan Cryptography

Bidang	Contoh
Web Security	HTTPS / SSL-TLS
Aplikasi	Login & enkripsi password
Jaringan	VPN
Ethical Hacking	Password cracking (hash analysis)
Blockchain	Digital signature & hash

Contoh Soal :

Soal 1.

Pesan berikut telah dienkripsi menggunakan Caesar Cipher dengan pergeseran **3 huruf**. Temukan flag-nya!

Iodj{Sbwxmrk_lv_hgmxxvqvo!}

Pemecahan

Gunakan Python untuk mendekripsi dengan Caesar Cipher (pergeseran -3):

```
def caesar_decrypt(ciphertext, shift):
    decrypted_text = ""
    for char in ciphertext:
        if char.isalpha():
            shift_base = ord('a') if char.islower() else ord('A')
            decrypted_text += chr((ord(char) - shift_base - shift) % 26 + shift_base)
        else:
            decrypted_text += char # Keep non-alphabet characters unchanged
    return decrypted_text

ciphertext = "Iodj{Sbwxmrk_lv_hgmxxvqvo!}"
print(caesar_decrypt(ciphertext, 3))
```

Output:

```
Flag{Python_is_awesome!}
```

Jawaban: Flag{Python_is_awesome!}

Soal 2.**Tujuan:**

Memecahkan pesan terenkripsi

Contoh: Caesar Cipher

```
def caesar(text, shift):  
    result = ""  
    for c in text:  
        if c.isalpha():  
            result += chr((ord(c)-97+shift)%26+97)  
        else:  
            result += c  
    return result  
  
cipher = "khoor"  
for i in range(26):  
    print(i, caesar(cipher, i))
```

Digunakan untuk brute force enkripsi sederhana

Kesimpulan

Cryptography berfungsi untuk menjaga keamanan, keaslian, dan kepercayaan data dalam sistem digital.

C. Web Exploitation (Simple Flask App)

1. Pengertian Web Exploitation

Web Exploitation adalah proses memanfaatkan celah keamanan (vulnerability) pada aplikasi atau server web untuk memperoleh akses tidak sah, data sensitif, atau kendali sistem, baik untuk tujuan pengujian keamanan (ethical hacking) maupun serangan siber.

Web exploitation biasanya dilakukan setelah tahap web reconnaissance dan vulnerability scanning.

2. Fungsi Web Exploitation

Dalam konteks **keamanan dan ethical hacking**, web exploitation memiliki beberapa fungsi utama:

a. Mengidentifikasi Celah Keamanan Web

Membantu menemukan kelemahan seperti:

- SQL Injection
- Cross-Site Scripting (XSS)
- File Inclusion (LFI/RFI)
- Broken Authentication

b. Menguji Keamanan Aplikasi Web

Digunakan oleh:

- **Penetration Tester**
- **Ethical Hacker**
- **Security Analyst**

Untuk memastikan aplikasi aman sebelum digunakan publik.

c. Mendapatkan Akses Tidak Sah (Simulasi)

Dalam lab atau pengujian:

- Bypass login
- Akses database
- Upload web shell

⚠ *Hanya diperbolehkan di sistem milik sendiri atau dengan izin resmi.*

d. Meningkatkan Sistem Pertahanan

Hasil exploitation digunakan untuk:

- Menutup celah keamanan
- Menerapkan patch
- Meningkatkan konfigurasi server

e. Pembelajaran Keamanan Web

Web exploitation berfungsi sebagai:

- Media pembelajaran praktis cyber security
- Simulasi serangan dunia nyata

Contoh Teknik Web Exploitation

Teknik	Fungsi
SQL Injection	Mengakses database
XSS	Menyisipkan script berbahaya
CSRF	Manipulasi aksi user
File Upload Exploit	Upload backdoor
Command Injection	Eksekusi perintah OS

Kesimpulan

Web Exploitation adalah teknik untuk menguji dan memahami kelemahan keamanan web agar sistem dapat diperbaiki dan diamankan.

Conto Soal :

Soal 1.

Sebuah aplikasi Flask memiliki endpoint tersembunyi yang berisi flag. Berikut adalah kode Python-nya:

```
from flask import Flask, request
```

```
app = Flask(__name__)
```

```
@app.route('/')
```

```
def home():
```

```
    return "Welcome to CTF!"
```



```

@app.route('/flag')
def flag():
    user_agent = request.headers.get("User-Agent")
    if user_agent == "hacker_browser":
        return "Flag{Python_Web_Exploitation}"
    else:
        return "Access Denied!"

if __name__ == "__main__":
    app.run(debug=True)

```

Tugas:

1. Temukan endpoint yang mengandung flag.
2. Kirim request dengan **User-Agent** yang benar.

Pemecahan

Gunakan **cURL** atau **requests** di Python untuk mengubah **User-Agent**:

```

import requests

url = "http://127.0.0.1:5000/flag"
headers = {"User-Agent": "hacker_browser"}

response = requests.get(url, headers=headers)
print(response.text)

```

Output:

```
Flag{Python_Web_Exploitation}
```

Jawaban: Flag{Python_Web_Exploitation}

D. Forensics (Extracting Hidden Data from Image)

1. Pengertian Forensics (Extracting Hidden Data from Image)

Forensics (Digital Forensics) adalah ilmu dan teknik untuk mengumpulkan, menganalisis, dan menyajikan bukti digital yang berasal dari perangkat elektronik seperti komputer, smartphone, server, jaringan, dan media penyimpanan, dengan tujuan mendukung investigasi insiden keamanan atau kejahatan siber.

Digital forensics dilakukan secara ilmiah, terstruktur, dan dapat dipertanggung jawabkan secara hukum.

2. Fungsi Forensics

Digital forensics memiliki beberapa fungsi utama, yaitu:

a. Mengumpulkan Bukti Digital

Mengambil data tanpa mengubah isi aslinya, seperti:

- Log sistem
- File yang terhapus
- Email
- Riwayat browser

b. Menganalisis Insiden Keamanan

Mengetahui:

- Cara serangan dilakukan
- Waktu kejadian
- Alat atau malware yang digunakan

c. Mengidentifikasi Pelaku

Membantu melacak:

- Alamat IP
- Akun pengguna
- Aktivitas mencurigakan

d. Mendukung Proses Hukum

Bukti digital dapat digunakan di:

- Pengadilan
- Audit keamanan

- Investigasi internal

e. Meningkatkan Keamanan Sistem

Hasil analisis forensik digunakan untuk:

- Menutup celah keamanan
- Mencegah serangan serupa
- Menyusun kebijakan keamanan

Contoh Penerapan Digital Forensics

Bidang	Contoh
Cyber Crime	Investigasi hacking
Perusahaan	Audit insiden kebocoran data
Jaringan	Analisis traffic mencurigakan
Mobile	Forensik WhatsApp & SMS
Ethical Hacking	Post-incident analysis

Kesimpulan

Forensics berfungsi untuk menemukan kebenaran dari bukti digital secara sah dan akurat.

Contoh Soal:

Soal 1.

File gambar **secret.png** memiliki flag tersembunyi di dalam metadata-nya. Temukan flag tersebut.

Pemecahan

Gunakan **exiftool** atau **Pillow (PIL) + Piexif** untuk membaca metadata:

```
exiftool secret.png
```

Atau gunakan Python:

```
from PIL import Image
import piexif

img = Image.open("secret.png")
```

```
exif_data = img.info  
print(exif_data)
```

Jika terdapat metadata yang mencurigakan seperti:

```
Exif: {'UserComment': b'Flag{Hidden_in_metadata}'}
```

Jawaban: Flag{Hidden_in_metadata}

Soal 2.

Tujuan:

Menganalisis file & data tersembunyi

Contoh: Membaca File

Misal isi flag.txt: RkxBR3toaWRkZW5fZmxhZ30=

Solusi :

```
import base64  
  
with open("flag.txt", "r") as f:  
    data = f.read()  
  
decoded = base64.b64decode(data)  
print(decoded.decode())
```

Jawaban: Flag{hidden_flag}

Mencari string tersembunyi

```
import re  
data = open("dump.txt").read()  
print(re.findall("CTF{.*?}", data))
```

E. Pwn (Python Buffer Overflow)

1. Pengertian

Pwn adalah istilah dalam keamanan komputer dan ethical hacking yang merujuk pada eksploitasi kelemahan program tingkat rendah, terutama memory corruption seperti buffer overflow, untuk mengambil alih kendali program atau sistem.

Python Buffer Overflow bukan berarti Python mudah overflow, karena:

- Bahasa Python bersifat memory-safe
- Buffer overflow umumnya terjadi pada program C/C++

Python digunakan sebagai alat (tool) untuk:

- Mengotomasi exploit
- Membuat payload
- Berinteraksi dengan program rentan (binary exploitation)

Contoh tool Python yang sering dipakai: pwntools.

2. Fungsi Pwn (Python Buffer Overflow)

Dalam konteks **ethical hacking dan keamanan sistem**, fungsi Pwn adalah:

a. Menguji Keamanan Program

Mengetahui apakah aplikasi (biasanya C/C++) memiliki celah seperti:

- Stack buffer overflow
- Heap overflow
- Format string vulnerability

b. Mengambil Kendali Alur Program (Control Flow)

Eksplorasi dilakukan untuk:

- Mengubah return address
- Menjalankan shell
- Mendapatkan akses sistem (simulasi)

⚠ *Hanya dilakukan di lab atau sistem dengan izin resmi.*

c. Automasi Eksploitasi dengan Python

Python digunakan untuk:

- Mengirim input berbahaya
- Membuat payload secara dinamis

- Remote exploitation (CTF / lab)

d. Pembelajaran Keamanan Sistem Operasi

Pwn membantu memahami:

- Cara kerja memory
- Stack & heap
- Proteksi modern (ASLR, NX, PIE)

e. Meningkatkan Pertahanan Sistem

Hasil pwn digunakan untuk:

- Menambahkan proteksi compiler
- Menggunakan safe function
- Hardening sistem

Contoh Konsep dalam Pwn

Konsep	Fungsi
Buffer Overflow	Menimpa memori
Return Address	Mengontrol eksekusi
Shellcode	Menjalankan perintah
ASLR / NX	Proteksi memori
Pwntools (Python)	Automasi exploit

Kesimpulan

Pwn (Python Buffer Overflow) adalah teknik eksploitasi kelemahan memori program menggunakan Python sebagai alat bantu untuk memahami, menguji, dan meningkatkan keamanan sistem.

Contoh Soal :

Soal 1.

Sebuah program Python menerima input panjang, dan jika inputnya cukup panjang, flag akan muncul.

```
def vulnerable_function():
    buffer = input("Enter input: ")
```

```
if len(buffer) > 50:
    print("Flag{Buffer_Overflow_Detected}")
else:
    print("Try again!")

vulnerable_function()
```

Pemecahan

Masukkan input yang sangat panjang (misalnya, 51 karakter):

```
python3 -c 'print("A"*51)' | python3 challenge.py
```

Output:

```
Flag{Buffer_Overflow_Detected}
```

Jawaban: Flag{Buffer_Overflow_Detected}

F. Brute Force Password

1. Pengertian

Brute Force Password adalah metode percobaan login dengan mencoba berbagai kombinasi password secara berulang-ulang hingga menemukan password yang benar.

Metode ini tidak memanfaatkan celah teknis khusus, tetapi mengandalkan:

- Banyaknya percobaan
- Password yang lemah
- Sistem tanpa pembatasan login

Tanpa izin, brute force termasuk tindakan ilegal. Dalam ethical hacking, teknik ini hanya digunakan untuk pengujian keamanan (penetration testing) pada sistem yang diizinkan.

2. Fungsi Brute Force Password

Dalam konteks keamanan sistem dan pembelajaran, brute force memiliki beberapa fungsi berikut:

a. Menguji Kekuatan Password

Mengetahui apakah password pengguna:

- Terlalu pendek
- Mudah ditebak
- Menggunakan pola umum

b. Mengidentifikasi Sistem yang Lemah

Menguji apakah sistem memiliki:

- Tidak ada limit percobaan login
- Tidak ada CAPTCHA
- Tidak ada mekanisme logout

c. Evaluasi Kebijakan Keamanan

Digunakan untuk menilai:

- Kebijakan password
- Mekanisme autentikasi
- Sistem monitoring login

d. Pembelajaran Keamanan Informasi

Membantu siswa memahami:

- Pentingnya password kuat
- Risiko password lemah
- Perlunya autentikasi berlapis

e. Meningkatkan Sistem Pertahanan

Hasil pengujian brute force digunakan untuk:

- Menerapkan account lockout
- Menggunakan MFA (Multi-Factor Authentication)
- Menambahkan rate limiting

Contoh Konsep Terkait Brute Force

Konsep	Keterangan
Password Weakness	Password mudah ditebak
Rate Limiting	Membatasi percobaan login
Account Lockout	Mengunci akun sementara
CAPTCHA	Mencegah percobaan otomatis
MFA	Keamanan berlapis

Kesimpulan

Brute Force Password berfungsi sebagai metode pengujian untuk mengetahui seberapa kuat sistem autentikasi dan mendorong penerapan keamanan yang lebih baik.

Contoh Brute Force Sederhana

```
passwords = ["1234", "admin", "password", "ctf123"]
```

```
for pwd in passwords:
```

```
    if pwd == "ctf123":
```

```
        print("Password ditemukan:", pwd)
```

Hanya untuk simulasi lab

G. Tools Python untuk CTF

Library	Fungsi
requests	Web attack
hashlib	Hash cracking
base64	Decode
re	Regex
pwntools	Binary exploitation

Contoh Soal CTF (Latihan)

Soal 1: Decode

```
Q1RGX3B5dGhvbG==
```

Petunjuk: Base64

```
import base64
print(base64.b64decode("Q1RGX3B5dGhvbG=="))
```

Soal 2: Brute Force PIN

```
for i in range(1000, 10000):
    if i == 4321:
        print("Flag ditemukan:", i)
```

Etika & Legalitas CTF

Boleh:

- Lab CTF
- Server simulasi
- Capture The Flag platform

Dilarang:

- Sistem nyata tanpa izin
- Website publik

Platform CTF untuk Belajar

- PicoCTF

- TryHackMe
- Hack The Box (Academy)
- Root-Me

Kompetensi yang Dicapai

Setelah materi ini, siswa mampu:

- Menggunakan Python untuk CTF
- Menganalisis masalah keamanan
- Membuat script sederhana ethical hacking
- Memahami dasar cybersecurity

Lanjutan (Opsional)

- Automasi exploit
- Python + Kali Linux
- Integrasi Bash & Python
- CTF berbasis jaringan

Kesimpulan

CTF berbasis Python sering menguji pemahaman tentang:

- Reverse Engineering → Analisis kode obfuscate.
- Cryptography → Menyelesaikan cipher sederhana.
- Web Exploitation → Menggunakan Flask untuk eksploitasi HTTP.
- Forensics → Menemukan data tersembunyi dalam file.
- Pwn → Mengeksploitasi kelemahan seperti buffer overflow.